

Assessment

CO1 - AT2

Name : Ch. Bhavya Sri

Reg no : 192411249

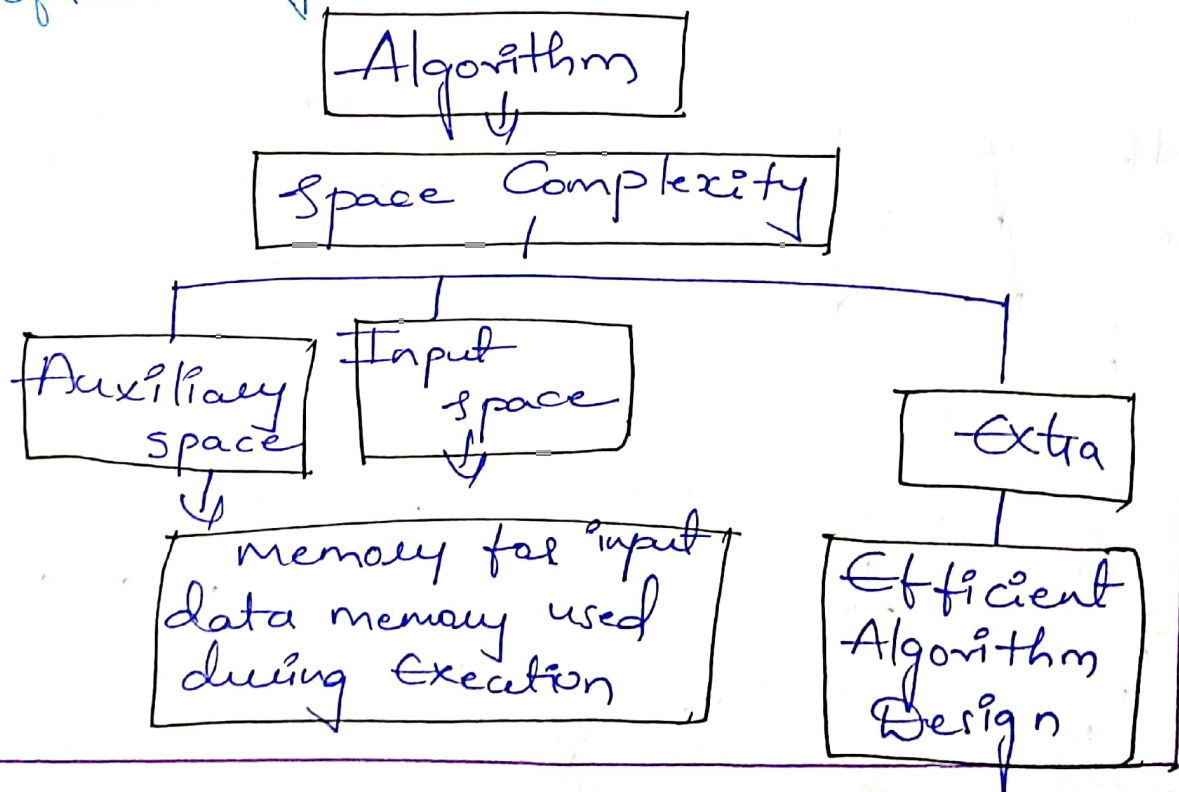
Course : CSA0612
Code

Course : Design and analysis
algorithms

Q23) Concept Map: Algorithm $\rightarrow$ Space Complexity
 $\rightarrow$ Auxiliary space and Input space $\rightarrow$
Total Memory Requirement

Introduction:

Space Complexity is the total amount of memory required by an algorithm during execution. It includes both the memory occupied by the input data and the additional memory used by the algorithm. Space Complexity is an important factor in algorithm analysis because many applications have limited memory resources. Efficient memory usage improves the overall performance of software systems.



Explanation of Each Node

1. Algorithm's

An algorithm is a finite sequence of instructions designed to solve a particular problem. Every algorithm requires both execution time and memory. Memory analysis helps determine whether an algorithm is suitable for systems with limited resources.

2. Space Complexity:

Space Complexity measures the total memory required by an algorithm for execution. It is represented as

$$\text{Space Complexity} = \text{Input Space} + \text{Auxiliary Space}$$

The goal is to minimize unnecessary memory usage while maintaining good performance.

3. Input Space:

Input Space refers to the memory required to store the input data and supplied to the algorithm.

Examples:

→ Array of n elements → $O(n)$

→ Matrix of $n \times n$ elements → $O(n^2)$

→ String of length n → $O(n)$

Input space depends only on the size of the input and cannot usually be reduced.

⇒ Auxiliary Space:

Auxiliary space is the extra memory used by an algorithm excluding the input.

Examples include:

Auxiliary space

→ Temporary variables

→ Stack memory

→ Queue

→ Recursive call stack

→ Dynamic memory allocation.

Auxiliary space depends on the algorithm design.

5. Total Memory Requirement

The total memory required is obtained by combining:

→ Input Space

→ Auxiliary Space

Formula:

$$\text{Total Memory} = \text{Input Space} + \text{Auxiliary Space}$$

Example:

If input array requires $O(n)$ memory and recursion requires $O(\log n)$,

$$\text{Total Memory} = \text{Input Space} + \text{Auxiliary Space}$$

Example:

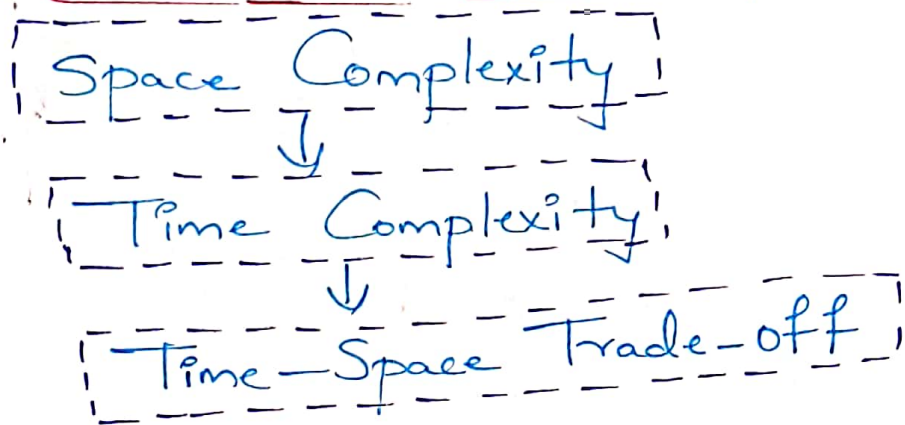
If Input array requires $O(n)$ memory and recursion requires $O(\log n)$.

$$\text{Total Memory} = \text{Input Space} + \text{Auxiliary Space}$$

$$\text{Total Space} = O(n) + O(\log n)$$

$$\text{Overall Complexity} = O(n)$$

Cross-Link with Time Complexity



Advantages:

- Better memory management
- Efficient algorithm design
- Prevents memory overflow
- Helps Compare algorithms
- Useful for embedded systems
- Improves Software Optimization.

Conclusion:

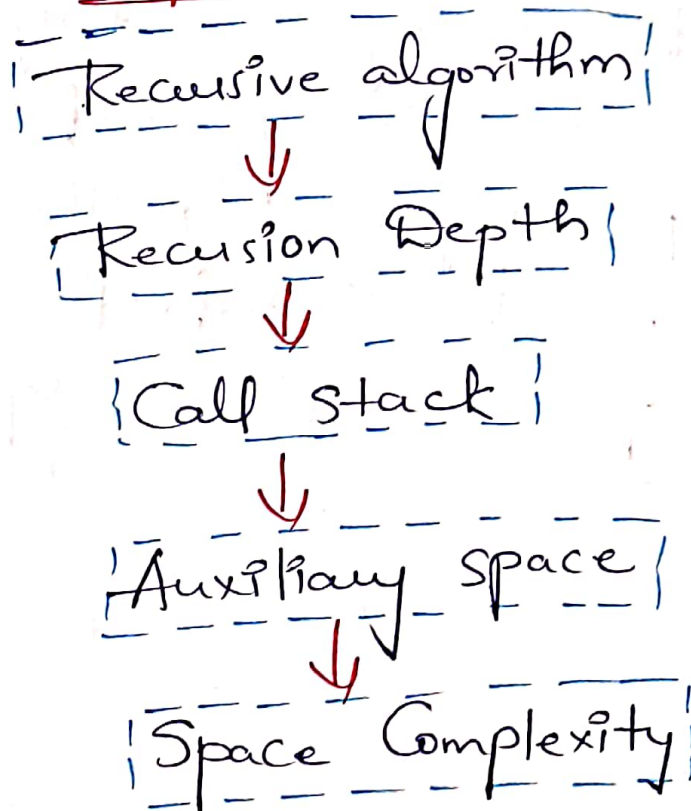
The Concept map clearly shows that Space Complexity consists of input space and auxiliary space. Together they determine the total memory requirement. Understanding the relationship between memory usage and execution

Q/24

Introduction :

A recursive algorithm solves a problem by repeatedly calling itself until a base condition is reached. Every recursive call creates a new stack frame in memory. Therefore recursion directly affects auxiliary space and overall space and complexity.

Concept map :



⇒ Explanation of each Node

1. Recursive Algorithm

A recursive algorithm calls itself to solve smaller instances of the same problem.

Every recursive algorithm contains:
→ Base Case → Recursive Case

Examples:

→ Factorial

→ Fibonacci

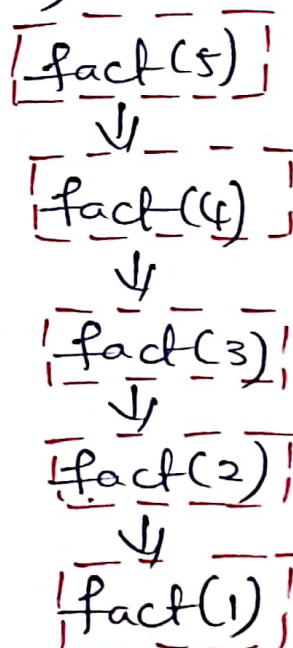
⇒ Recursion Depth

Recursion depth means recursive function calls the base case.

the number of before reaching

Example:

Factorial (5)



Depth = 5

3. Call stack

Each recursive call creates a stack frame.

Each frame stores:

- Function parameters
- Local Variables
- Return address

Comparison with iterative Algorithm

